R3D-Planner

Historische Entwicklung und technische Baseline

Versionsevolution, ROS 2-Integration und Ausgangszustand für die zukünftige Distribution im ROS 2-Ökosystem

12. Oktober 2026

Historische Ausgangsdokumentation für ein Forschungs- und Entwicklungsprojekt zur ROS 2-Konformität und öffentlichen Bereitstellung des R3D-Planners.

Repository: RoboProjekt/R3D-Planner
Referenzbranch: main
Technische Baseline:

`c65642fe56015556f696364a724cb5b976a27888`

Konsolidierung der elf Dokumente der abgeschlossenen Historienanalyse. Keine Implementierung von ROS-bezogenen Änderungen.

Inhaltsverzeichnis

1. Einleitung 4
1.1 Zweck und Forschungsbezug . . . 4
1.2 Untersuchungsgrenze und Evidenz . . . 4
1.3 Definition der technischen Baseline . . . 4

2. Repository-Überblick 5
2.1 Pakete und technische Grundlage . . . 5
2.2 Struktur des Referenzstands . . . 5
2.3 Datenfluss und Systemgrenze . . . 5
2.4 Branches und historische Bestandsaufnahme . . . 6

3. Versionsgeschichte 7
3.1 Import und gemeinsame Vorgeschichte . . . 7
3.2 V1: duale Repräsentation als historischer Zweig . . . 7
3.3 V2: bewusste Beschränkung auf PCD . . . 8
3.4 V3: Konfiguration und standardisierte Pose-Eingabe . . . 8
3.5 main: Kommentarheader ohne Verhaltensänderung . . . 9
3.6 Verzweigungen, Äquivalenz und zeitliche Grenzen . . . 9

4. Funktionale Entwicklung 10
4.1 R3D-Preprocessor . . . 10
4.2 R3D-Planner . . . 11
4.3 ROS-Funktionsgrenze . . . 11
4.4 Navigationskomponenten und Roboterauswahl . . . 12

5. Architektonische Entwicklung 13
5.1 Vom Workspace-Abbild zu zwei Quellpaketen . . . 13
5.2 Von parallelen Persistenzwegen zum PCD-Vertrag . . . 13
5.3 Modularisierung der Konfiguration und Pose-Eingabe . . . 13
5.4 Komposition und weiterhin externe Zuständigkeiten . . . 13

6. ROS 2-Integrationsgeschichte 14
6.1 Paketmetadaten, Build und Installation . . . 14
6.2 Nodes und öffentliche Schnittstellen . . . 14
6.3 Action-, Odometry- und TF-Vertrag . . . 15
6.4 Launch, Parameter und Eingabevalidierung . . . 16

7. Tests und Qualitätssicherung 17
7.1 Entwicklung und Belegstatus . . . 17
7.2 Dokumentierte Ergebnisse und unabhängige Prüfungen . . . 18
7.3 Integration, Umgebung und Fehlerbehandlung . . . 18
7.4 Verbleibende Qualitätssicherungsgrenzen . . . 18

8. Abhängigkeiten und Lizenzierung 19
8.1 Metadaten und Dependency-Änderungen . . . 19
8.2 Komponenten- und Lizenzinventar . . . 19
8.3 Entwicklung der Notices und Verteilungsfragen . . . 20

9. Versionsvergleich 21

10. Initialzustand des Forschungsvorhabens 22
10.1 Verbindlicher Referenzstand . . . 22
10.2 Bestands- und Forschungszuordnung . . . 22
10.3 Zuordnungsregeln und offene Belege . . . 23

11. Identifizierter Entwicklungsbedarf 24
11.1 Bestehender Stand, Defizit und geplante Änderung . . . 24

11.2 Verifikationsbedarf und Qualitätsspielraum . . . 25
11.3 Abhängigkeitsgerechte Reihenfolge . . . 25

12. Schlussfolgerung 26

Anhang A. Commit-Übersicht und Chronologie 27

Anhang B. Reproduzierbarkeit und Branch-Nachweise 29

B.1 Vergleichsregeln . . . 29
B.2 Stabile Patch-IDs und Äquivalenzen . . . 29

Anhang C. Quellpfade und Funktionsnachweise 30

Anhang D. Quellenkonsolidierung und Grenzen 32

D.1 Primärdokumente und inhaltliche Zuordnung . . . 32
D.2 Administrative Bestandsinformationen . . . 32
D.3 Umfang der Dokumentoperation . . . 33

1. Einleitung

1.1 Zweck und Forschungsbezug

Dieser Bericht rekonstruiert die in R3D-Planner belegte Entwicklung und definiert den Ausgangszustand für das Forschungsvorhaben zur Weiterentwicklung für eine zukünftige ROS 2-Distribution. Er beantwortet, welche Algorithmen, Schnittstellen, Paketstrukturen und Qualitätssicherungsmaßnahmen bereits vor dem Vorhaben existierten. Diese Leistungen sind vom noch erforderlichen Distributions- und Konformitätsaufwand zu trennen.

Grundlage sind elf bereits abgeschlossene Markdown-Dokumente. Ihre überlappenden Befunde wurden thematisch zusammengeführt, Begriffe vereinheitlicht und die Chronologie in eine zusammenhängende Darstellung überführt. Die vorliegende Konsolidierung wiederholt keine vollständige Git-Analyse. Sie ergänzt weder Funktionalität noch Abhängigkeiten und führt keine neuen ROS-Laufzeit-, Build- oder Hardwaretests aus.

1.2 Untersuchungsgrenze und Evidenz

Der Gegenstand ist ausschließlich das Repository `RoboProjekt/R3D-Planner` einschließlich seiner Branches V1, V2, V3 und main. Unabhängige Vorgänger-/Nachfolgeprojekte, externe Forks und andere Repositories sind ausgeschlossen. In älteren R3D-Dokumenten genannte externe Deployments gelten hier nur als historische Aussagen, nicht als eigenständig untersuchte Implementierungen.

Kennzeichnung	Bedeutung im Bericht
Verifiziert	Durch Git-Objekte, Quellcode oder Diffs der zugrunde liegenden Historienanalyse belegt. Die PDF-Konsolidierung übernimmt diese Nachweise; sie behauptet keine erneute Laufzeitprüfung.
Dokumentiert	In damaliger Projektdokumentation oder archivierten Ausgaben berichtet; Ausführung oder externe Sachlage nicht unabhängig reproduziert.
Abgeleitet	Technische Interpretation eines belegten Sachverhalts; von der Implementierungsaussage zu unterscheiden.
Nicht bestimmbar	Die verfügbaren Unterlagen erlauben keine belastbare Feststellung.

Tabelle 1: Evidenzstufen. Commit-Kürzel [C01] bis [C13] werden in Anhang A vollständig aufgelöst.

Eine vorhandene Codefunktion beweist weder ihre allgemeine Korrektheit noch sichere Bewegung auf realer Hardware. Branch-Namen sind keine semantischen Releases oder Quality Levels. Die erste Git-Erwähnung eines importierten Algorithmus ist nicht automatisch dessen ursprüngliches Entwicklungsdatum.

1.3 Definition der technischen Baseline

Die verbindliche technische Referenz ist main bei `c65642fe56015556f696364a724cb5b976a27888` [C13]. Der Referenzstichtag ist der 12. Oktober 2026; der Commit selbst stammt vom 17. September 2026 um 00:56:32 Uhr +02:00. Implementierungsdatum und Auditdatum bleiben getrennt. Die vollständige Bestands- und Forschungszuordnung steht konzentriert in Kapitel 10.

Die Historienanalyse umfasst 13 eindeutige Commits in vier Branches. Es gibt einen Root-Commit, keine Merge-Commits und keine Tags oder veröffentlichten Releases. In den zugänglichen Branches ist kein Nachfolger des Baseline-Commits belegt. Drei ältere V1-Dokumentationscommits außerhalb der main-Abstammung sind daher keine Änderungen nach der Baseline [C04] [C07] [C08] [C11].

2. Repository-Überblick

2.1 Pakete und technische Grundlage

Das Projekt enthält zwei Python-Pakete: `r3d_preprocessor` verarbeitet eine vorhandene Punktwolkenkarte zu einer begehbarkeitsklassifizierten Repräsentation; `r3d_planner` rekonstruiert daraus einen Graphen und stellt Pfadplanung sowie ROS-nahe Navigationskomponenten bereit. Beide verwenden ament_python mit setuptools, format-3-Manifeste und colcon als dokumentiertes Workspace-Werkzeug [C01] [C12]. CMake-/C++-Targets und eigene Message-, Service- oder Action-Definitionen sind in der untersuchten kanonischen Quelle nicht vorhanden.

Die Dokumentation nennt Ubuntu 22.04 beziehungsweise 22.04.5 LTS und ROS 2 Humble als Entwicklungs-/Deploymentziel. Ein geprüfter Wechsel auf Jazzy oder neuere Distributionen und eine Multi-Plattform-Matrix sind nicht belegt. Die generische Verwendung von rclpy, ROS-Nachrichten, launch und TF stellt hierfür keinen Testnachweis dar [C04] [C10] [C12].

2.2 Struktur des Referenzstands

```
R3D-Planner/
  README.md, INSTALL.md, README_r3d_planner.txt
  docs/                 Architektur, Konfiguration, bekannte Probleme, Berichte
  r3d_planner/
    package.xml, setup.py, setup.cfg, resource/r3d_planner
    config/             planner_config.yaml, robots/Go2W.yaml, Legacy-Nav2-YAML
    launch/             planner, preprocessor, map, follower
    r3d_planner/        Planner, RViz, Filter, Follower, Konfigurations-/Pose-Helfer
    test/               Funktionstests und drei Linter-Templates
  r3d_preprocessor/
    package.xml, setup.py, setup.cfg, resource/r3d_preprocessor
    r3d_preprocessor/   PCD-Analyse und Publisher
    maps/               Eingabe-PCD und analysierte RGB-PCD
    test/               drei Linter-Templates
```

Am technischen Referenzstand liegen 59 versionierte Dateien und 26 Python-Dateien vor. V1/V2/V3 enthalten 52/47/59 Dateien beziehungsweise 21/18/26 Python-Dateien. Diese Zahlen beziehen sich auf die Git-Snapshots, nicht auf später erzeugte Auditdateien. Verbliebene __pycache__-Artefakte, die unvollständige Legacy-Nav2-Konfiguration und ein alternativer Publisher ohne Console-Entry sind von der aktiven Anwendung zu unterscheiden [C03] [C05] [C12] [C13].

2.3 Datenfluss und Systemgrenze



Abbildung 1: Aktive PCD-Verarbeitung und Navigation. Hardwaretreiber, Odometriefrontend und globale Lokalisierung liegen außerhalb des Repositories [C12].

Die Pipeline ist offline kartenbasiert. Das Projekt erzeugt keine Live-Karte und implementiert kein vollständiges SLAM-Frontend. Der Follower erhält den globalen Pfad und lokale Hindernispunkte; separate Klippen-/Treppenmeldungen sind nicht in seine

Bewegungskontrolle eingebunden. Eine ROS-Action-Schnittstelle allein macht die Implementierung nicht zu einem vollständigen Nav2-Planner-Plugin.

2.4 Branches und historische Bestandsaufnahme

Branch	Tip / Referenz	Datum (+02:00)	Rolle
V1	01348f3 [C11]	15.09.2026 16:02:04	Dualer Pickle-/PCD-Zweig; sieben erreichbare Commits.
V2	3abd002 [C10]	15.09.2026 16:01:50	PCD-only; acht erreichbare Commits.
V3	3752404 [C12]	16.09.2026 15:08:56	Konfiguration/Odometry; neun erreichbare Commits.
main	c65642f [C13]	17.09.2026 00:56:32	V3-Funktionalität plus Kommentarheader; zehn erreichbare Commits.

Die Vereinigung dieser Historien enthält 13 Commits; die Einzelzahlen dürfen nicht addiert werden. Lokal ist main vorhanden, die weiteren Branches liegen als origin/V1, origin/V2 und origin/V3 vor. origin/HEAD ist lediglich ein Alias. Der konfigurierte origin verweist per SSH auf RoboProjekt/R3D-Planner. Die frühere Analyse dokumentiert übereinstimmende Remote-Tips und keine registrierten zugänglichen Forks; für dieses PDF erfolgte keine erneute externe Untersuchung. Weitere Bestandsdetails stehen in Anhang D.

3. Versionsgeschichte

3.1 Import und gemeinsame Vorgeschichte

Der früheste belegbare Zustand im untersuchten Repository ist der Import vom 14. September 2026, 16:44:14 Uhr [C01]. Er umfasst 770 Dateien in einem verschachtelten ros2_r3d_planner_ws samt build-, install- und log-Ausgaben, Karten und Pickle-Graphen. Die kanonischen src-Verzeichnisse enthalten bereits beide Verarbeitungspipelines, Traversierbarkeitsanalyse, A*, RViz, lokale Filterung und Pfadverfolgung. Ursprungsdaten vor diesem Import sind nicht bestimmbar. Frühere Datumsangaben in übernommenen Log-Verzeichnisnamen sind kein Beleg für eine bestimmte Quellcodeeinführung.

Elf Minuten später ersetzt [C02] den Workspace durch flache Pakete im Repository-Root. Dabei entfallen generierte Workspace-Ausgaben, Wurzelartefakte und das Shell-Startskript. Zugleich fehlen vorübergehend PCD-Analyzer, PCD-Planner, TF-Helfer und alternativer Publisher; RGB-Packung und Marker-Alpha werden zurückgenommen. Der Commitname „Added newest Version“ belegt deshalb keine monotone funktionale Erweiterung.

Um 17:00 Uhr stellt [C03] diese Komponenten wieder her. Die frühere Analyse verglich 31 kanonische Textdateien mit dem Root-Snapshot: Nach Entfernung des Workspace-Präfixes sind sie byteidentisch. „Added PCD support“ ist somit eine Wiederherstellung im flachen Layout, keine erstmalige Entwicklung des PCD-Workflows. RGB-Darstellung, Shutdown-Guard und Marker-Alpha sind gegenüber dem Zwischenstand wieder korrigiert, aber bereits im Root vorhanden.

Am selben Abend ergänzt [C04] README, INSTALL, Architektur, bekannte Probleme und README-Audit und erweitert die Paketdokumentation. Dieser Stand ist der gemeinsame Verzweigungspunkt für V1 und V2. Die vollständige Commit-Chronologie mit Parent-Beziehungen steht in Anhang A.

3.2 V1: duale Repräsentation als historischer Zweig

Referenz: origin/V1, `01348f3285a0e7582f70cc5f2ed3d1bf4b7f904b` [C11], 15.09.2026 16:02:04 +02:00, Autor Basti. Direkter Parent ist [C08]; der gemeinsame Vorläufer der beiden Entwicklungslinien ist [C04]. V1 ist kein getaggtes Release.

V1 hält die Pickle- und die PCD-Repräsentation parallel vor. Der Pickle-Preprocessor speichert Graph, Ursprung, Voxelgröße, Roboterradius und Hindernisse; der alternative Analyzer schreibt klassifizierte Weltkoordinaten in eine RGB-PCD. Beide Planner bieten KDTree-Snapping, A*, Pfad- und Marker-Ausgabe sowie ComputePathToPose. RViz und Follower beziehen die Live-Pose über TF.

Die drei V1-eigenen Commits [C07], [C08] und [C11] schreiben Dokumentation um, gestalten das README als Nutzungsanleitung und anonymisieren Beispielpfade. Sie verändern keine ausführbare Quelle, keine Tests und keine Abhängigkeiten. Es gibt sechs Planner- und vier Preprocessor-Console-Scripts, aber keine Python-Launchdateien am V1-Tip. Das frühere Full-Stack-Shellskript war bereits durch [C02] entfernt.

Grenzen sind der ignorierte use_start-Schalter, ungeprüfte Eingabeframes, unbegrenztes Nearest-Node-Snapping und die Kodierung des narrow-Merkmals in einer nicht normierten Quaternion. Der Follower publiziert bei ausbleibender TF-Pose keinen neuen Nullbefehl. Vorhandene Linter-Templates und ein manueller TF-Helfer sind keine funktionale Testsuite. Paketmetadaten und Lizenzbelege sind unvollständig.

3.3 V2: bewusste Beschränkung auf PCD

Referenz: origin/V2, `3abd002e8cc9e7a26cacaf43a5c20b1822664f14` [C10], 15.09.2026 16:01:50 +02:00, Autor Basti. Die Linie entsteht ab [C04] über [C05], [C06] und [C09]. Der finale V1-Tip ist kein Vorfahr von V2; er liegt zeitlich sogar 14 Sekunden später.

Die eigentliche Architekturänderung erfolgt in [C05]: Pickle-Generator (399 Zeilen), Pickle-Planner (211 Zeilen), Pickle-Marker-Publisher (114 Zeilen), zugehöriger Bytecode und drei Console-Scripts werden entfernt. Die verbleibenden ausführbaren PCD-Quellen sind gegenüber V1 unverändert. V2 führt folglich weder einen neuen A*-Algorithmus noch neue Boden-/Treppenanalyse ein. Es macht einen bereits vorhandenen Weg zum alleinigen aktiven Workflow.

Das Preprocessor-Manifest verliert visualization_msgs, std_msgs, geometry_msgs und pickle. Der Pfad Roh-PCD → analysierte RGB-PCD → rekonstruierter Graph wird in den Anleitungen konsistent dargestellt. [C06] und [C09] betreffen Dokumentationsstil und Nutzungsanleitung; [C10] ergänzt den Referenzbericht und korrigiert dokumentierte Integrations-/Sicherheitsannahmen. Solche Textkorrekturen sind keine Laufzeit-Bugfixes.

Die geerbten Action-, TF-, Cloud- und Controllergrenzen bleiben bestehen. Es gibt weiterhin keine zentrale Konfiguration, direkte standardisierte Odometry-Subscription oder Python-Launchdateien. Die sechs Linter-Templates bleiben; funktionale Tests kommen nicht hinzu. Der Referenzbericht enthält dokumentierte Lint- und Deploymentaussagen, deren Status Kapitel 7 erläutert.

3.4 V3: Konfiguration und standardisierte Pose-Eingabe

Referenz: origin/V3, `3752404a08867e9e6f3725300b88e148d4d5be37` [C12], 16.09.2026 15:08:56 +02:00, Autor Basti. Direkter Parent ist der V2-Tip [C10]. Alle substantiellen V3-Erweiterungen liegen in diesem einen Commit.

V3 trennt gemeinsame Konfiguration, Launch-Aufbau und Odometry-Eingabe in configuration.py, launching.py und odometry.py. planner_config.yaml wählt eine Robot-YAML; Geometrie, gemeinsame Raster-/Schrittparameter und komponentenbezogene Einstellungen werden validiert und an vorhandene Nodes weitergeleitet. Vier installierte Launchdateien starten Analyse, Planner mit optionalem Filter/RViz, Kartenanzeige und separat den Follower.

Der Planner berücksichtigt nun use_start: Ein expliziter Start bleibt für Offline-Planung möglich, andernfalls wird eine aktuelle Odometry-Pose mittels zeitgestempelter map/odom-TF in den Kartenframe transformiert. Falsche Ziel-/Startframes und fehlende oder veraltete Online-Pose führen zum Abbruch. RViz delegiert die Online-Startwahl an den Planner. Der Follower publiziert bei Poseverlust Nullgeschwindigkeit. Legacy-Kalibrierungs-TF kann zugunsten eines externen Eigentümers deaktiviert werden.

Kostenmultiplikatoren, Darstellungsoptionen, Filter-/Follower-Tuning und ausgewählte Topics/Frames werden parameterisiert. Sechs Runtime-Abhängigkeiten, Installationsregeln für Launch-/Robot-Ressourcen und 20 fokussierte Testfälle kommen hinzu. Analyzer, PCD-Repräsentation, Topologie, A* und planarer Regler werden dabei nicht neu entwickelt. Ein dokumentierter erfolgreicher V3-Testlauf ersetzt keine unabhängige Hardwarevalidierung.

Die Live-Source-/Symlink-Konfigurationspolitik bleibt eine Installationsgrenze: Normale kopierte YAML-Snapshots werden ohne R3D_CONFIG_DIR abgelehnt. Karten werden nicht installiert. Absolute interne Namen, Cloud-Layout-Annahmen, Kalibrierungsmathematik, narrow-Quaternion und fehlende Controller-Anbindung der Klippen-/Treppenausgänge bleiben offen. V3 implementiert kein Mapping-Frontend und keinen vollständigen Nav2-Plugin-Vertrag.

3.5 main: Kommentarheader ohne Verhaltensänderung

Referenz: main/origin/main, `c65642fe56015556f696364a724cb5b976a27888` [C13], 17.09.2026 00:56:32 +02:00, Autor Basti; direkter Parent ist V3. Der Diff fügt 18 Python-Dateien jeweils zehn Kommentarzeilen hinzu, insgesamt 180 Zeilen ohne Löschung. Die frühere Analyse bestätigt gleiche ASTs aller geänderten Dateien.

Die Header nennen Autor, Repository, Last-modified-Datum und Copyright. Sie enthalten keinen neuen Lizenzgrant. Laufzeitfunktionalität, Architektur, Abhängigkeiten und Testassertionen entsprechen V3. main ist daher die dokumentierte Baseline, kein belegtes neues nummeriertes Funktionsrelease.

3.6 Verzweigungen, Äquivalenz und zeitliche Grenzen

```
ca9f449 -> ce4af0b -> a24a3d3 -> 253ff81
                                 |-- 7a4c0cf -> 0306f8f -> 01348f3  V1
                                 |-- 337bbd3 -> 3e5360a -> cf0c527
                                                           -> 3abd002  V2
                                                              -> 3752404  V3
                                                                 -> c65642f main
```

Zwischen V1 und V2 stehen drei beziehungsweise vier eindeutige Commits; gegenüber main hat V1 drei, main sechs eindeutige Commits. V2→V3 und V3→main sind jeweils direkte Erweiterungen um einen Commit. Gleichlautende Dokumentationssubjects haben verschiedene stabile Patch-IDs; eine patchidentische Cherry-Pick-Übernahme wurde nicht nachgewiesen. Manuelles Kopieren ist nicht bestimmbar. Anhang B bewahrt die Patch-IDs und Reproduktionskommandos. Shared Commits werden nicht als doppelte Feature-Einführung gezählt.

4. Funktionale Entwicklung

4.1 R3D-Preprocessor

Die erste belegbare Quelle liegt bereits in [C01]; die aktuelle PCD-Variante ist r3d_preprocessor/r3d_preprocessor/r3d_pcd_analyser.py. VoxelTraversabilityAnalyzer trennt Laden/Voxelisierung, Bodenaugmentation, Clearance-Prüfung, Flächenfüllung, Treppenmarkierung und Export. Der AST dieser Analyse bleibt zwischen Wiederherstellung [C03], V2, V3 und main gleich. Die V3-Änderung betrifft ihre Konfiguration, nicht den Algorithmus.

4.1.1 Punktwolke, Voxel und Bodenverarbeitung

load_and_voxelize liest mit Open3D eine vorhandene Punktwolke und weist leere/nicht gefundene Eingaben zurück. Die Voxelindizes entstehen relativ zum Open3D-Grid-Ursprung; Belegungen werden anhand der Punktanzahl pro Voxel gefiltert. Die standardmäßige Voxelgröße beträgt 5 cm.

augment_floor_from_density zerlegt XY in Analysezellen, sortiert die Höhen und trennt Cluster bei einer ausreichend großen Höhenlücke. Ein flaches Cluster wird bei $z_range < 2 \cdot floor_height_tolerance + 0{,}02$ m als Grundlage dichter Zusatzpunkte verwendet. Die effektive Mindestpunktzahl je Zelle lautet $\max(3, \text{int}(min_points_per_sqm \cdot analysis_grid_size^2))$. Bei 0,20 m Raster führen etwa Werte 1 und 10 beide zur Untergrenze drei. Die Toleranz bildet also nicht selbst die Cluster, sondern prüft deren Planarität [C01] [C03] [C10].

Zusatzpunkte liegen auf der Cluster-Mittelhöhe in halber Voxelweite. Anschließend füllt _fill_floor_planes unterstützte Nachbarlücken iterativ unter erneuter Clearance-Prüfung. Iterationszahl, Suchradius und minimale Nachbarzahl sind vorhandene Analyseparameter. Diese Interpolation ist keine Live-Kartierung und beweist nicht die Begehbarkeit eines physisch unbekannten Bereichs.

4.1.2 Traversierbarkeit, Hindernisse und Treppen

check_collision_cylinder prüft belegte Voxel im zylindrischen Robotervolumen oberhalb der Base-Clearance. identify_walkable_nodes versucht zunächst den normalen Radius und danach einen kleineren narrow-Radius. Im Beispiel liegen Radius/narrow-Radius/Höhe/Base-Clearance bei 0,40/0,30/0,80/0,10 m. Geometrische Parameter existieren schon im Import; V3 selektiert und übermittelt sie aus der Robot-YAML [C01] [C12].

identify_stairs vergleicht begehbare Höhen in acht benachbarten XY-Spalten. Differenzen oberhalb der minimalen und bis einschließlich der maximalen Stufenhöhe markieren stair_access. Standardgrenzen sind 5 und 25 cm. Das belegt geometrische Klassifikation, keine Richtungsspezifikation oder getestete Treppenlokomotion. Nicht begehbare belegte Voxel werden als Hindernisse exportiert.

PCD-Klasse	Farbe (RGB)	Bedeutung
floor	0 / 1 / 0	Normal begehbarer Boden.
narrow	0 / 1 / 1	Nur kleinerer Clearance-Radius frei.
stair_access	1 / 0,8 / 0	Nachbarübergang im Stufenhöhenbereich.
obstacle	1 / 0 / 1	Belegt und nicht als begehbar eingestuft.

Tabelle 2: Impliziter Farbvertrag zwischen save_colored_pcd und PCD-Planner, vorhanden seit [C01].

save_colored_pcd schreibt Voxelmittelpunkte in Weltkoordinaten mit Klassenfarben und dem Suffix _analysed.pcd. Voxelgröße, Geometrie, Fülloptionen und Schwellenwerte werden nicht als Sidecar/Metadaten gespeichert. Eine andere Roboterauswahl verlangt deshalb eine erneute Analyse. Geometrieparameter im Planner ändern eine schon exportierte Klassifikation nicht.

4.2 R3D-Planner

4.2.1 Repräsentation und Graphrekonstruktion

Der historische Pickle-Weg lädt einen gespeicherten NetworkX-Graphen und rekonstruiert Weltpositionen aus Ursprung + Index · Voxelgröße + halber Voxelgröße. Der PCD-Weg in r3d_planner/r3d_planner/r3d_pcd_path.py liest Weltkoordinaten und Farben, überspringt magenta Hindernisse und leitet Integer-Keys durch round(Koordinate / Voxelgröße) ab. Cyan wird narrow, gelb stair_access; andere nicht magenta Farben fallen auf floor zurück [C01] [C03].

Die Rekonstruktion verbindet acht benachbarte XY-Spalten und zulässige Höhen. Der Graph ist ungerichtet; Nodes enthalten pos/type, Edges Gewichte. Die reale ursprüngliche Graphmetadatenpersistenz entfällt mit V2. Unbekannte Farben, gerundete doppelte Keys und abweichende Rasterparameter werden nicht durch einen vollständigen Dateiformatvertrag abgefangen. Die gemeinsame V3-Konfiguration mindert Parameterdrift, löst aber keine Prüfung einer fremden/alten PCD.

4.2.2 A*, Kosten und Pfaderzeugung

a_star verwendet einen Heap, g_score und came_from; heuristic berechnet euklidische 3D-Distanz. Ein Closed-Set beziehungsweise die gezielte Verwerfung veralteter Heap-Einträge ist nicht implementiert. Es wurde kein neuer Suchalgorithmus zwischen V1 und Baseline eingeführt. Eine umfassende Optimalitäts-/Performance-Regression über reale Karten ist nicht dokumentiert.

Kanten tragen geometrische 3D-Länge. Flache Verbindungen erhalten bei narrow-Endpunkten standardmäßig Faktor 1,2, Stufenverbindungen Faktor 3,0. V3 ersetzt Literale durch narrow_cost_multiplier und stair_cost_multiplier; der zentrale Loader fordert mindestens 1. Robotervolumen wird beim Preprocessing berücksichtigt, nicht bei jedem geladenen Planner-Node erneut geprüft [C12].

get_nearest_node projiziert Start und Ziel per SciPy-KDTree auf den nächsten begehbaren Punkt, ohne maximalen Abstand. execute_callback erzeugt nav_msgs/Path, Action-Ergebnis und Marker. Der PCD-Planner visualisiert als LINE_STRIP, der frühere Pickle-Planner als CUBE_LIST. Für narrow-Posen werden z=1 und w=1 in der Orientierung gesetzt; die Quaternion ist nicht normiert und dient intern als Flag. Generische Pose-Konsumenten erhalten damit keinen sauberen Orientierungsvertrag.

4.3 ROS-Funktionsgrenze

Standardnachrichten, ComputePathToPose, Trigger, RViz-Eingaben und Pfad-/Kartenpublikationen sind bereits im importierten Funktionsumfang vorhanden. V3 ergänzt eine standardisierte Odometry-Eingabe und parameterisierte Integrationspunkte, ohne einen vollständigen Nav2-Lifecycle-/Plugin-Vertrag herzustellen. Das konkrete Interface-Inventar, TF-Eigentum und Action-Verhalten sind in Kapitel 6 gebündelt; Anhang C ordnet Funktionen den Quellpfaden zu.

4.4 Navigationskomponenten und Roboterauswahl

ObstacleCliffFilter liest den Cloud-Bytebuffer als dicht gepacktes XYZ-float32-Layout. Felder, point_step, row_step, Endianness und organisierte Anordnung werden nicht vollständig ausgewertet. Seine Grenzwerte liegen unmittelbar im Sensorframe; eine Transformation nach base_link erfolgt nicht. Höhen-/Distanzfilter bilden Hindernisse, ein ROI erkennt Stufenkandidaten, eine geringe Bodendichte führt zu einer virtuellen Klippenwand [C01] [C12].

PathFollower implementiert standardmäßig 10 Hz planare Lookahead-/Heading-Regelung mit Begrenzung der Winkelgeschwindigkeit. Er nutzt XY sowie das narrow-Flag; Pfad-Z und stair_access steuern keine spezielle Lokomotion. Er stoppt bei fehlendem Pfad, Hindernis oder erreichtem Ziel, seit V3 auch bei fehlender Pose. Klippenwand und stair_detect werden intern nicht abonniert; Hindernisfrische und ein genereller Command-Watchdog fehlen.

Die Standardkombination aus 0,65 m Mindestzuverlässigkeitsradius im Filter und 0,6 m Stop-Fenster im Follower kann einen zentralen Hindernisbereich von der konsumierten Ausgabe ausschließen. Das ist ein quellcodegestützter Konfigurationsbefund, kein gemessener Vorfall. V3 macht die Grenzwerte einstellbar, korrigiert ihre gesamte Systemwirkung aber nicht automatisch.

Die Robot-YAML besitzt Name, vier Geometriewerte, Odometry-Topic/Frames, Sensortopic und cmd_vel-Ausgang. Go2W ist ein vorhandenes Beispiel, kein Beweis universeller Roboterkompatibilität. Ein alternativer Dateiname lässt sich ohne Quelländerung auswählen, sofern Nachrichten-, Frame-, Sensor- und Controllerverträge eingehalten werden. Der manuelle PathTestTFNode publiziert lediglich statische odom/base-TF und liefert keine Odometry-Nachricht für V3.

5. Architektonische Entwicklung

5.1 Vom Workspace-Abbild zu zwei Quellpaketen

Der Root vereint kanonische Quelle und historische Ausgaben. Generated build/lib-Dateien können gegenüber src veraltet sein; installierte Entry-Points beweisen daher nicht die Konsistenz jeder mitkopierten Quelle. [C02] entfernt das Workspace-Abbild und lässt zwei flache ROS-Pakete zurück. Technisch wird dadurch die Grenze zwischen Projektquelle und lokalem Buildzustand deutlicher. Verbleibender Bytecode bedeutet, dass die Bereinigung nicht vollständig ist.

5.2 Von parallelen Persistenzwegen zum PCD-Vertrag

V1 kann einen vorab gebauten Graphen mit ausgewählten Metadaten laden oder ihn aus klassifizierten Punkten rekonstruieren. V2 entfernt bewusst den ersten Weg [C05]. Die zweite Möglichkeit ist dabei keine Neuentwicklung. Abgeleitet sinkt die Zahl parallel zu pflegender Werkzeuge; visuell prüfbare PCD ersetzt Python-Objektpersistenz. Gleichzeitig geht die explizite Herkunft von Graphparametern verloren: Eine PCD ist ohne passende Einstellungen nicht vollständig reproduzierbar.

Die öffentliche Repräsentation umfasst nun Koordinaten, Klassenfarben und separat konfigurierte Raster-/Schrittparameter. Topologie und Kosten entstehen beim Plannerstart. Damit hängt die Reproduktion sowohl vom Kartenartefakt als auch vom verwendeten Konfigurationssatz ab. Diese technische Bedeutung ist von einer rechtlichen oder umfassenden Sicherheitsbewertung des Dateiformats zu trennen.

5.3 Modularisierung der Konfiguration und Pose-Eingabe

V3 führt drei gemeinsame Helfermodule ein [C12]. configuration.py übernimmt Dateiauflösung, Struktur-/Wertprüfung und Parameternormalisierung; launching.py wählt Nodes und übermittelt Konfiguration; odometry.py kapselt Posevalidierung und Transformation. Der Kern von Preprocessing, Graphaufbau, Suche und Regler bleibt erhalten. Die Modularisierung betrifft somit Integrationszuständigkeiten, nicht den Ersatz des Planners.

Die zentrale YAML definiert gemeinsame Raster-/Stufengrenzen genau einmal. Roboterspezifische Geometrie und Ein-/Ausgangsverträge werden in einer gewählten Robot-Datei geführt. Sensorlokale Stufenschwellen bleiben absichtlich getrennt von Offline-Graphgrenzen, weil Koordinatensysteme und Semantik unterschiedlich sind. Abgeleitet vermindert dies Duplikation und Auswahländerungen im Sourcecode, garantiert aber keine korrekten physikalischen Grenzwerte.

5.4 Komposition und weiterhin externe Zuständigkeiten

Der Plannerlaunch setzt optionale Wahrnehmungs-/RViz-Komponenten zusammen; Analyse, Kartenanzeige und Follower bleiben separat startbar. Ein übergeordneter vollständiger Hardware-/Lokalisierungslaunch ist nicht vorhanden. Das ist ein vorhandener modularer Einstieg, dessen künftige Hierarchie nach realen Nutzerabläufen beurteilt werden sollte, kein zwingender Anlass für einen kompletten Neuaufbau.

Odometry und globale TF bilden seit V3 eine explizite Integrationsgrenze. Der Adapter sendet keine TF; externe Odometrie und globale Lokalisierung behalten ihre Zuständigkeit. Command-Transport, Watchdog, Mux und Notstopp sind keine in diesem Repository belegten Implementierungen. Diese Abgrenzung verhindert, dass vorhandene Detektions- oder Planungsfunktionen mit vollständiger Systemnavigation gleichgesetzt werden.

6. ROS 2-Integrationsgeschichte

6.1 Paketmetadaten, Build und Installation

Bereits der Import [C01] enthält format-3-package.xml, setup.py/setup.cfg, resource-Marker, Python-Modulverzeichnisse und Console-Entry-Points beider Pakete. Das Manifest exportiert ament_python; setup.cfg installiert Executables nach $base/lib/<Paket>. Planner-YAML wird schon damals über data_files installiert. Diese ROS-Paketgrundlage ist eine vorhandene Leistung, keine erst noch zu schaffende Forschungsfunktion.

Die flache Organisation entsteht [C02], ausgewählte PCD-Entry-Points werden [C03] wiederhergestellt. V2 entfernt drei Legacy-Scripts [C05]. V3 installiert zusätzlich config/robots/*.yaml und launch/*.launch.py [C12]. Die Karten fehlen weiterhin in den data_files des Preprocessors. Manifeste sind syntaktisch lesbar, enthalten aber keine buildtool_depend-Einträge und bleiben fachlich unvollständig; XML-Parsebarkeit beweist keine rosdep-/Release-Eignung.

Versionen, Beschreibungen und Lizenzdeklarationen entwickeln sich nicht parallel zu den Branch-Labels: package.xml bleibt 0.0.0 mit TODO-Beschreibung/Lizenz, setup.py bleibt 0.0.1 und nennt Apache-2.0. Diese Unterschiede werden in Kapitel 8 zusammen mit der Abhängigkeitsgeschichte bewertet.

6.2 Nodes und öffentliche Schnittstellen

Die folgende Übersicht bündelt die ROS-Verträge. Bis auf die direkten Odometry-Subscriptions existieren die genannten aktiven Schnittstellen schon im Root [C01], werden im flachen Layout [C03] wiederhergestellt und in V3 teilweise parameterisiert [C12]. Angaben betreffen Implementierung, nicht einen gemessenen ROS-Graphen. Topic-Publisher/-Subscriber verwenden Tiefe 10, soweit kein anderes Profil angegeben ist.

Node / Executable	ROS-Vertrag und Rolle
pcd_to_graph_node / pcd_analyser	Offline-Analyse ohne Subscriber/Publisher; schreibt analysierte PCD. Bestehende Geometrie-/Analyseparameter werden seit V3 zentral übermittelt.
global_graph_planner / pcd_path_planner	Publiziert /global_path (nav_msgs/Path) und /planned_path (visualization_msgs/Marker). Relative Action compute_path_to_pose (nav2_msgs/ComputePathToPose). Seit V3 konfigurierbares nav_msgs/Odometry-Topic, Standard /lidar_odometry.
obstacle_cliff_filter / local_filter	PointCloud2-Eingang, ursprünglich /hesai_ros_driver/hesai/lidar_points, seit V3 wählbar. Ausgänge /local/filtered_obstacles und /local/cliff_virtual_wall (PointCloud2), /stair_detect (PointStamped).
r3d_path_follower / path_follower	Abonniert /global_path und /local/filtered_obstacles, seit V3 auch Odometry. Publiziert geometry_msgs/Twist; cmd_vel_topic ist seit V3 wählbar, Standard /cmd_vel.
r3d_rviz_interface / rviz_interface	Eingänge /clicked_point (PointStamped), /initialpose (PoseWithCovarianceStamped), /goal_pose (PoseStamped). Relative Trigger-Service recalibrate_pose; absoluter Action-Client /compute_path_to_pose; optional statische map/odom-TF.
pcd_publisher / pcd_server	Publiziert /map_pointcloud (PointCloud2), Tiefe 1, transient-local; RGB-Packung seit Root/Wiederherstellung, map_frame seit V3 wählbar.
r3d_path_test / path_test	Statische Identitätstransformation odom→base_link. Keine Odometry-Nachricht und kein automatischer Assertion-Test.

Tabelle 3: Aktive beziehungsweise manuelle ROS-Einstiegspunkte. Vollständige Quellpfade und Symbole: Anhang C.

Historisch nutzte global_planner denselben Node-Namen global_graph_planner und denselben Action-/Pfadvertrag auf Pickle-Basis. voxel_map_publisher publizierte /r3d_global_voxel_map als Marker mit Tiefe 1 und transient-local. Beide entfallen in V2. Der verbliebene r3d_pcd_voxel_publisher.py hat keinen Console-Entry, verwendet aber wie pcd_server den Node-Namen pcd_publisher und /map_pointcloud; er liefert XYZ statt aktivem XYZRGB und behält einen festen map-Frame.

Die Action ist serverseitig relativ, clientseitig in RViz absolut. Viele Pfad-, RViz- und Filtertopics sind absolut; die Robotkonfiguration validiert volle absolute Topicnamen. Remapping ist damit nicht ausgeschlossen, aber eine konsistente Namespace-Komposition nicht vollständig entworfen. Gemischte Node-Namenspräfixe sind ein Konventionsbefund, kein pauschaler Release-Verstoß. Eigene ROS-IDL-Typen werden nicht eingeführt.

6.3 Action-, Odometry- und TF-Vertrag

Vor V3 setzt RViz aus einem map/base-TF-Lookup einen expliziten request.start und use_start=true. Der Follower liest map/base-TF unabhängig. Der Planner ignoriert den use_start-Schalter und verarbeitet numerische Start-/Zielkoordinaten ohne Frameprüfung, während er den Goal-Frame in die Ausgabe übernimmt. Diese Einschränkung ist im Source belegt, nicht bloß aus dem Actionnamen abgeleitet [C01] [C03].

Seit V3 kapselt OdometryInput die Eingabe [C12]. callback prüft odom_frame und base_frame, endliche Position und nutzbare Quaternion. map_pose vergleicht den Zeitstempel mit der Node-Uhr: Negative Zeitdifferenz oder Überschreitung der Standardfrist von 1 s führen zum Fehler. Der Adapter sucht map←odom zum Nachrichtenzeitpunkt, rotiert/verschiebt die Position und komponiert die Orientierung. Die Subscription nutzt qos_profile_sensor_data. Der Adapter sendet selbst keine TF.

Die dokumentierte Systemzuständigkeit lautet: Externe Odometry liefert odom→base_link-Pose und gegebenenfalls die zugehörige TF; globale Lokalisierung besitzt map→odom. Mathematisch verwendet der Adapter die Odometry-Nachricht und map/odom-TF. Eine Sensorframe-Pose muss extern auf den konfigurierten Base-Frame umgerechnet werden; ein bloßes Umbenennen des Topics genügt nicht. Optional publiziert RViz weiterhin seine Legacy-map/odom-Kalibrierung, sofern publish_map_odom nicht false ist.

execute_callback prüft den Goal-Frame und bei use_start=true auch den expliziten Startframe gegen map_frame. Bei false wird die validierte Odometry-Pose verwendet. Fehlende/ungültige/veraltete Pose oder fehlende globale TF führen zum Abbruch. Arbiträre Zielframes werden nicht transformiert. RViz setzt nun use_start=false; der Follower publiziert bei nicht verfügbarer Pose Null-Twist. Diese konkreten Vertragskorrekturen stammen vollständig aus [C12].

Die Action bleibt begrenzt: planner_id wird nicht ausgewertet, Cancellation-Handling ist nicht implementiert, Planning-Time und weitere Ergebnisdaten werden nicht vollständig gesetzt. Die narrow-Orientierung bleibt intern zweckentfremdet. recalibrate_pose setzt nur den lokalen RViz-Zustand zurück. Das direkte Einsetzen der angeklickten Pose als map/odom-Transform berücksichtigt eine bereits von null verschiedene Odometriepose nicht vollständig; dies ist ein geerbter mathematischer Integrationsvorbehalt.

6.4 Launch, Parameter und Eingabevalidierung

Launch	Gestartete Komponenten	Konfiguration
preprocessor.launch.py	pcd_analyser, einmalige Analyse	Robotgeometrie, shared, preprocessing, Eingabekarte.
planner.launch.py	pcd_path_planner; optional unabhängig local_filter/RViz	Planner-, Robot-, Filter-/RViz-Einstellungen; vier Enable-Kombinationen.
map.launch.py	pcd_server	Analysierte Karte und Kartenframe.
follower.launch.py	path_follower, ausdrücklich separat	Odometry-/cmd_vel-Vertrag und Controllertuning.

Tabelle 4: V3-Launchstruktur, installiert durch [C12].

configured_nodes/description in launching.py nutzt OpaqueFunction und IfCondition. load_configuration in configuration.py liest planner_config.yaml und eine relative Robot-Datei innerhalb des Konfigurationsverzeichnisses. Es prüft fehlende/unbekannte/duplizierte Keys, YAML-Mappingstruktur, numerische Typen, endliche Werte, Dimensionsbeziehungen, Boolesche Flags, Topic-/Framenamen und ausgewählte PCD-Dateien. Für ROS-Doubleparameter normalisiert es Zahlen auch dann, wenn YAML sie ganzzahlig schreibt.

Die zentrale Konfiguration führt shared-Parameter für Voxel-/Schritthöhen genau einmal. Geometrie, Odometry, Sensor- und Command-Topics liegen im Robotprofil; allgemeine Analyse-, Planner-, Filter-, RViz- und Reglerwerte liegen zentral. Das geerbte r3d_planner_params.yaml ist dagegen ein unvollständiges, nicht von den neuen Launches geladenes Nav2-Fragment. Es belegt weder einen vollständig laufenden Nav2-Stack noch die Nutzung des darin genannten Controllers.

default_config_path priorisiert lebende Quelldateien oder Symlinks und verwirft normale kopierte Installations-YAML ohne R3D_CONFIG_DIR. Beispielkarten werden über benachbarte Source-Verzeichnisse adressiert; ihre Installation fehlt. Direkter Node-Aufruf umgeht zudem die zentrale Validierung. Launchargumente, Namespace-Selektor, automatischer Hardware-/Lokalisierungsstart und ein vollständiger Systemlaunch sind nicht vorhanden. Das dokumentiert eine verbesserte Entwicklungsintegration mit verbleibenden Portabilitätsgrenzen.

7. Tests und Qualitätssicherung

7.1 Entwicklung und Belegstatus

Die Qualitätssicherung beginnt nicht erst bei V3. Seit dem Import bestehen in jedem Paket test_copyright.py, test_flake8.py und test_pep257.py samt Testabhängigkeiten und pytest-Setup. Copyrighttests sind mit skip versehen, auch nach den main-Headern. Der manuelle path_test sendet nur TF; sein Name macht ihn nicht zum Unit-/Integrationstest [C01] [C13].

Erst [C12] ergänzt test_configuration.py mit neun Top-Level-Testfunktionen, die durch Parametrisierung 20 Fälle ergeben. Ein einzelner realer Restart-Test durchläuft intern vier Varianten; diese werden nicht zusätzlich als vier gesammelte pytest-Fälle gezählt. Es sind pytest-basierte ROS-Prozesstests, keine launch_testing-Suite.

Testfunktion / Gruppe	Fälle	Geprüfte Funktion
test_selection_and_restart_reload	1	Alternatives Robotprofil; Reload und Weitergabe geänderter Geometrie, Topics und shared-Werte.
test_validation	9	Fehlende Robot-/Höhenwerte, unzulässige Radien/Höhen/Topics/Boolean-Werte, duplizierte/fehlerhafte YAML und ungültige Voxelgröße.
test_launch_conditions	4	IfCondition aller unabhängigen Filter-/RViz-Kombinationen; Planner bleibt bedingungslos.
test_effective_ros_parameters_and_odometry	1	Produktionskonstruktoren mit Overrides, synthetische Odometry-Rezeption, Translation/Rotation, Online-/Explicit-Start und Stale-Ablehnung.
Source-/Install-/Typprüfungen	3	test_default_live_source_config; test_copied_install_does_not_silently_load_snapshot; test_integer_yaml_spelling_keeps_ros_double_types.
test_preprocessor_real_launch	1	ros2 launch auf kleiner temporärer PCD; Exitcode und Existenz der analysierten Ausgabedatei.
test_real_launch_restart_and_all_components	1	Vier echte Restarts, geänderte YAML/Robotwahl, erwartete Node-Mengen und CLI-Parameterwerte ohne erneuten Build/Source.

Tabelle 5: Vorhandene fokussierte V3-Tests [C12]. Summe: 20 Fälle; keine Aussage über einen neuen Lauf im Rahmen der PDF-Erstellung.

Der Odometry-Integrationstest ruft den Action-Callback mit einem Fake-Goal-Handle auf, nicht über eine vollständige Action-Transporttransaktion. Die Preprocessor-Launchprüfung bestätigt eine Ausgabedatei, aber nicht alle Terrainklassifikationen. Die Aussagekraft der Tests ist daher konkret, nicht flächendeckend.

7.2 Dokumentierte Ergebnisse und unabhängige Prüfungen

Nachweisebene	Vorhandene Aussage	Einordnung
Historischer V1/V2-Bericht [C10]	PEP257 bestanden, Copyright übersprungen; V2-Flake8: 151 Preprocessor-/166 Planner-Befunde; V1: 286/200. Workstation-Build durch lokale Packaging-Konflikte blockiert.	Dokumentiert; keine neue Lint-/Buildausführung. Lokaler Konflikt ist kein Nachweis universeller Buildunfähigkeit.
V3-Implementierungsbericht [C12]	20 Tests in 40,07 s unter Fast DDS/isolierter Humble-Umgebung; temporäre Symlink-Builds; fokussierte Flake8-Prüfung mit 100-Zeichen-Grenze.	Dokumentiert; Legacy-Lintprobleme verbleiben. Keine neue oder unabhängige Hardwarevalidierung.
Abgeschlossene Historienanalyse	Alle kanonischen Python-Dateien und beide XML-Manifeste an allen 13 Commits geparst; V3/main-AST-Gleichheit aller 18 geänderten Dateien; V1/V2-Sourcegleichheit.	Als statische Prüfung der Voranalyse verifiziert. Parserfolg beweist weder Laufzeit noch Depen dency-/Releasevollständigkeit.
Vorliegende PDF-Konsolidierung	Quellübernahme, Nachweis-/Strukturkontrolle, PDF-Text-/Seiten-/Layoutprüfung.	Dokumentprüfung; keine ROS-, colcon-, Lint- oder Hardwareausführung.

Tabelle 6: Implementierte Tests, berichtete Ausführung und unabhängige Prüfung sind getrennte Evidenz.

Die historische Formulierung, V3-Änderungen seien im ursprünglichen Implementierungstask nicht committed/gepusht worden, beschreibt dessen damaligen Kontext. Der Code samt Bericht ist im untersuchten [C12] enthalten. Ebenso belegt eine Aussage über ein zuvor getestetes Go2W-Deployment nicht automatisch V3-Hardwarevalidierung. Externe Deploymentfakten werden hier nicht neu untersucht.

7.3 Integration, Umgebung und Fehlerbehandlung

Manifeste nennen ament_copyright, ament_flake8, ament_pep257 und python3-pytest; setup.py nutzt tests_require. Damit besteht ein ament_python-/colcon-Testpfad, jedoch kein in dieser Konsolidierung unabhängig erfolgreicher colcon test/test-result-Lauf. Die Suite benötigt gesourcte ROS-Pakete, CLI, DDS und wissenschaftliche Bibliotheken. Temporäre PCD-/YAML-Fixtures vermeiden feste Entwicklerkartenpfade; Source-Konfiguration, exakte Nodenamen, isolierte Domain und POSIX-Prozessgruppen bleiben Umgebungsannahmen.

Schon Root prüft Dateien/leere Clouds und bricht erfolglose Pfadsuche ab. V3 ergänzt strikte Konfiguration, numerische Typnormalisierung, Pose-/Frame-/Zeitprüfung und den Nullbefehl bei Poseverlust. Diese vorhandenen Validierungsmaßnahmen sind ein wesentlicher Ausgangspunkt. Sie erstrecken sich nicht automatisch auf sämtliche direkten ROS-Parameterüberschreibungen.

7.4 Verbleibende Qualitätssicherungsgrenzen

Nicht vorhanden sind repräsentative Klassifikationsfixtures für Boden/narrow/Treppen/Hindernisse/Füllung, PCD-/Graph-Roundtrip samt Metadatenfehlern, A*-Kosten-/No-Path-/Grenzfallregressionen, maximale Snapping-Distanzen, vollständiger Action-Transport/Cancellation und systematische Cloud-Feld-/Stride-/Frametests. Follower-Poseverlust, frische/veraltete Hindernisse, Klippen-/Treppenintegration, kopierte Installationen und reale Hardwarebewegung sind nicht umfassend automatisch geprüft.

Nicht belegt sind CI/CD, Coverage, Typprüfung, Securityscan und Formatierungsautomation. CMake-/clang-Nachweise einschließlich ament_lint_auto/common sind für diesen Python-Bestand nicht einschlägig. Fehlende optionale Qualitätstools sind nicht pauschal zwingende ROS-Releasebedingungen.

8. Abhängigkeiten und Lizenzierung

8.1 Metadaten und Dependency-Änderungen

Feld	Beide Pakete / historische Entwicklung
Manifest / Buildtyp	Format 3, ament_python seit Root; Namen r3d_planner / r3d_preprocessor.
Versionsangaben	package.xml: 0.0.0; setup.py: 0.0.1, unverändert über alle untersuchten Stände.
Beschreibung / Maintainer	XML: TODO: Package description; Maintainer bauya, baau1001@stud.hs-kl.de. Keine später ergänzten Manifest-URL-/Author-Elemente.
Lizenzdeklaration	XML: TODO: License declaration; setup.py: Apache-2.0. Kein projektweiter LICENSE-/COPYING-Text.
Python-Setup	install_requires nur setuptools, tests_require pytest; kein vollständiger wissenschaftlicher Runtime-/Lock-Satz.
Testabhängigkeiten	ament_copyright, ament_flake8, ament_pep257, python3-pytest bereits seit Root.

Tabelle 7: Historisch unveränderte Metadatenbasis; Quellen: beide Manifeste und setup.py [C01] [C12] [C13].

Das Planner-Manifest deklariert seit Root rclpy, nav2_msgs, geometry_msgs, nav_msgs, sensor_msgs, visualization_msgs, scipy, numpy und std_srvs. Der Preprocessor nennt zunächst rclpy, visualization_msgs, std_msgs, geometry_msgs, sensor_msgs, numpy und pickle. V2 entfernt die vier Legacy-Einträge visualization_msgs, std_msgs, geometry_msgs und pickle [C05]. pickle ist Python-Standardbibliothek; die Manifestentfernung belegt keine Deinstallation einer externen Bibliothek.

V3 ergänzt beim Planner exec_depend für launch, launch_ros, ament_index_python, python3-yaml, tf2_ros und r3d_preprocessor [C12]. ament_index_python/tf2_ros werden teilweise schon vorher verwendet und erst jetzt explizit deklariert. Open3D/NetworkX sowie direkte std_msgs-/sensor_msgs_py-Nutzung bleiben in den betreffenden Manifesten unvollständig abgebildet. Erfolgreiche vollständige rosdep-Auflösung ist daraus nicht abzuleiten.

README/INSTALL beschreiben Ubuntu-/ROS-Systempakete, unter anderem NumPy, SciPy, NetworkX und Open3D, sowie einen rosdep-skip-keys-Workaround für numpy/scipy, in V1 zusätzlich pickle. Die Unterlagen warnen vor globalem pip-Mixing mit ROS Python. Es gibt keinen belegten gepinnten pip-Runtime-/Lock-Vertrag und keinen in dieser Konsolidierung verifizierten unmodifizierten Clean-Dependency-Lauf [C08] [C09] [C10] [C12].

8.2 Komponenten- und Lizenzinventar

Die folgende Tabelle gibt ausschließlich in R3D-Planner enthaltene Lizenzbelege wieder. Die Vertrautheit mit einer Bibliothek und deren allgemeine Verbreitung ersetzen keinen Quellenbeleg. Es erfolgt keine externe Lizenzprüfung und keine Kompatibilitätsentscheidung.

Komponente	Nutzung / Entwicklung	Lizenzbeleg im Repository / offene Frage
R3D-Funktionscode	Planner/Preprocessor seit Root; Konfiguration/Pose/Launch seit V3.	Apache-2.0 im Setup vs XML-TODO; kein Projektlizenztext. Autorisierung und Rechtekette nicht bestimmbar.
Ament-Testtemplates	Sechs Linter-Testdateien seit Root.	Apache-2.0-Notice und OSRF-Copyright in Dateien; gilt nicht automatisch für unabhängigen Funktionscode.

Komponente	Nutzung / Entwicklung	Lizenzbeleg im Repository / offene Frage
Open3D	PCD-Laden, Voxelisierung und Export/Display in beiden Paketen.	Lizenz nicht bestimmbar aus Repository; in beiden Manifesten nicht vollständig deklariert.
NetworkX	Graphstruktur, historische Persistenz; Planner weiterhin.	Lizenz nicht bestimmbar; direkte Planner-Abhängigkeit fehlt im Manifest.
NumPy / SciPy	Arrays/Geometrie bzw. KDTree seit Root.	Lizenzen nicht bestimmbar; rosdep-Schlüssel/Versionkompatibilität für Zielplattform zu prüfen.
PyYAML	Loader und Tests seit V3.	Lizenz nicht bestimmbar; python3-yaml als exec_depend vorhanden.
ROS-Bibliotheken/-Messages	rclpy, TF, Nachrichten, Nav2-Action, launch, ament_index_python, sensor_msgs_py.	Bibliothekslizenzen im Repository nicht bestimmbar. Deklarationslücken sind von Lizenzfragen zu trennen.
PCD-Karten / alte Graphen	Importierte Karten/Graphen; am Referenzstand zwei PCD-Artefakte.	Urheber, Datenherkunft und Weiterverteilungsrechte nicht bestimmbar.
Externe Hardware-/SLAM-Komponenten	In Dokumentation benannte Deploymentvoraussetzungen.	Keine SDK-/Frontend-Implementierung in den aktuellen Paketen gebündelt; externe Lizenzprüfung außerhalb des Umfangs.

Tabelle 8: Repository-interne Lizenzbelege und Untersuchungsgrenzen [C01] [C05] [C12] [C13].

Imports belegen Bibliotheksnutzung, nicht Vendoring ihres Quellcodes. Ein Open3D-, NetworkX- oder Hardware-SDK-Implementationstree ist in den kanonischen Paketen nicht identifiziert. Die historischen build/lib-Duplikate sind Projektkopien. Weitere kopierte/veränderte externe Algorithmusherkunft bleibt jenseits der erkennbaren Testtemplate-Notices nicht bestimmbar.

8.3 Entwicklung der Notices und Verteilungsfragen

Der Konflikt zwischen XML-Platzhalter und Apache-2.0 im Setup besteht bereits [C01] und bleibt über Layoutwechsel, V2 und V3 erhalten. Erst [C13] fügt 18 funktionalen/zugehörigen Python-Dateien Author-, Repository-, Last-modified- und Copyright-Header hinzu. Kein SPDX-License-Identifier oder Lizenzgrant kommt hinzu. Die September-14/16-Kommentardaten sind kein unabhängig belegtes Einführungsdatum, da die Header erst September 17 committed werden.

Git-Identität Basti beziehungsweise der Copyrightname Bastian Aumer beweisen keine vollständige Eigentums-/Einwilligungskette. Auch das in der Voranalyse dokumentierte GitHub-Lizenzfeld null ist keine rechtliche Entscheidung. Vor einer öffentlichen Weiterverteilung müssen Projektlizenz, Rechte am importierten Code, Kartenrechte und erforderliche Fremdnotices belastbar geklärt werden. Diese Fragen sind offen; der Bericht wählt oder appliziert keine Lizenz.

Technische Distributionshindernisse sind davon getrennt: Metadatenwidersprüche, fehlende direkte Dependencies, Quelllayout-Kartenpfade und die Beschränkung kopierter Konfigurationen verhindern den Nachweis eines reproduzierbaren installationsunabhängigen Pakets. Aus dieser historischen Zusammenfassung allein folgt keine abschließende normative ROS-Index-/Bloom-Entscheidung; aktuelle Zielkriterien bedürfen eigener Verifikation.

9. Versionsvergleich

Die Matrix verdichtet die Unterschiede, ohne die Funktionsbeschreibungen zu wiederholen. Present bedeutet im Source vorhanden; Partial bezeichnet eine konkrete Grenze; Absent bedeutet nicht vorhanden; Removed eine belegte Entfernung; Unknown unzureichende Evidenz. Keine Zelle ist ein Laufzeit-, Sicherheits- oder Quality-Level-Zertifikat.

Komponente / Feature	V1	V2	V3	main
Zwei ament_python-Pakete	Present	Present	Present	Present
Voxel-/Boden-/Clearance-Analyse	Present	Present	Present	Present
RGB-PCD und Graph/A*	Present	Present	Present	Present
Pickle-Generator/-Planner/-Marker	Present	Removed	Removed	Removed
ComputePathToPose-Server	Present	Present	Present	Present
use_start und Map-Frame-Validierung	Absent	Absent	Present	Present
Direkte Odometry / zeitgestempelte TF-Pose	Absent	Absent	Present	Present
RViz-Kalibrierung / Goal-Paarung	Present	Present	Present	Present
Externe map/odom-Eigentümerschaft wählbar	Absent	Absent	Present	Present
Lokale Obstacle-/Cliff-/Stair-Detektion	Present	Present	Present	Present
Planarer Follower / Twist	Present	Present	Present	Present
Nullbefehl bei Poseverlust	Absent	Absent	Present	Present
Zentrale YAML / Roboterauswahl	Absent	Absent	Present	Present
Installierte Python-Launchdateien	Absent	Absent	Present	Present
Kopierte Installations-/Ressourcenportabilität	Partial	Partial	Partial	Partial
Ament-Linter-Templates	Present	Present	Present	Present
Funktionale Config-/ROS-/Launchtests	Absent	Absent	Present	Present
CI/CD / Coverageautomation	Absent	Absent	Absent	Absent
Projektweiter geklärter Lizenzgrant	Unknown	Unknown	Unknown	Unknown
Neue Author-/Copyrightheader im Funktionscode	Absent	Absent	Absent	Present
Vollständige Cliff-/Stair-Motion / Watchdog	Absent	Absent	Absent	Absent
Live-Mapping / vollständiges Nav2-Plugin	Absent	Absent	Absent	Absent
Belegte Multi-Distribution-Kompatibilität	Unknown	Unknown	Unknown	Unknown

Tabelle 9: Exakte Branch-Tips aus Kapitel 2. Gemeinsame Features [C01] [C03], Entfernung [C05], Integration [C12], Header [C13].

Die wichtigsten Differenzen sind bewusst begrenzt: V2 entfernt einen Persistenzweg; V3 erweitert Konfiguration und ROS-Eingabeverträge; main ergänzt Notices. Der Algorithmuskern ist nicht pro Branch neu entstanden. Partial bei Installation verweist auf vorhandene Script-/Ressourceninstallation bei fehlenden Karten und eingeschränktem kopiertem Config-Betrieb. Unknown bei Lizenz und Distributionen bedeutet fehlenden Nachweis, nicht automatisch eine bestimmte Unverträglichkeit.

10. Initialzustand des Forschungsvorhabens

10.1 Verbindlicher Referenzstand

Repository: R3D-Planner / RoboProjekt/R3D-Planner. **Branch:** main. **Baseline-Commit:**

`c65642fe56015556f696364a724cb5b976a27888` [C13]

Referenzdatum: 12. Oktober 2026. **Commitdatum:** 17. September 2026, 00:56:32 +02:00. Der Commit definiert den technischen Ausgangspunkt; der spätere Stichtag datiert die Bewertung. Die Nachvollziehbarkeit darf weder durch einen anderen Branch noch durch den Zustand eines anderen Repositories ersetzt werden.

Die historische Analyse findet keinen späteren Baseline-Nachfolger in den vier zugänglichen Branches. V1 enthält drei nicht in main erreichbare, aber ältere Dokumentationscommits. Auch die Erstellung dieses PDF ist eine Dokumentationsleistung nach dem technischen Stand, keine Implementierung der geplanten Distributionsvorbereitung.

10.2 Bestands- und Forschungszuordnung

Alle in der Spalte „Bestehender Zustand“ genannten implementierten Fähigkeiten sind vor dem Forschungsbaseline-Stichtag vorhanden. Nachweise verweisen auf die Einführung oder maßgebliche Änderung im untersuchten Git-Verlauf; die Referenzdateien sind über Anhang C auffindbar.

Aspekt	Bestehender Zustand	Evidenz	Forschungsrelevanz
Preprocessing	Voxelisierung, Dichte-/Flächenfüllung, Clearance, floor/narrow/stair/obstacle, RGB-PCD.	[C01] [C03]	Bestehenden Algorithmus dokumentieren und prüfen; nicht als neu entwickelte Forschung klassifizieren.
Planner	PCD-Graph, KDTree, gewichtetes A*, Action-/Path-/Marker-Ausgabe; Pickle seit V2 entfernt.	[C01] [C05]	Verteilbarkeit und Verträge verbessern; kein grundlegender neuer Planner erforderlich.
Navigation	RViz-Paarung, lokaler Detektor, planarer Follower/Twist; geerbte Grenzen.	[C01] [C12]	Vorhandene Funktionen und unverbundene Ausgänge getrennt bewerten.
Paketstruktur	Zwei ament_python-Pakete, format-3-Manifeste, Marker, Scripts; flache Struktur.	[C01] [C02]	Vorhandene Grundlage standardisieren, nicht neu aufbauen.
Metadaten	0.0.0 vs 0.0.1; TODO-Lizenz/Beschreibung; Maintainer vorhanden.	[C01] [C13]	Konsistenz und belastbare Release-Metadaten herstellen.
Dependencies	ROS-/Science-Deklarationen; V2 reduziert, V3 erweitert; direkte Imports fehlen.	[C05] [C12]	Reproduzierbare Dependency-Auflösung und direkte Deklarationen nachweisen.
Launch	Vier installierte Launchdateien; Planner kombiniert optional Filter/RViz, Follower separat.	[C12]	Bestehende Komposition weiterverwenden; Nutzer-/Systemhierarchie bei Bedarf ergänzen.
Konfiguration	Zentrale YAML, Robotselector, shared-Parameter, Validierung und Typnormalisierung.	[C12]	Bereits vorhanden; künftige Arbeit betrifft Vertrags-/Installationsgrenzen.
Odometry / TF	Validierte Standard-Odometry, zeitgestempelte map/odom-Pose, expliziter Startmodus.	[C12]	Bestehende Integration absichern; kein neu zu entwickelndes Mapping-Frontend.

Aspekt	Bestehender Zustand	Evidenz	Forschungsrelevanz
ROS-Verträge	Action/Service/Topics; use_start/Frameprüfung und Poseverlust-Nullbefehl vorhanden.	[C01] [C12]	Namespaces, Metadaten und Action-Grenzen gezielt prüfen.
Installation	Ressourcen/Scripts installiert; Karten fehlen, kopierte Config absichtlich abgelehnt.	[C01] [C12]	Clean-Prefix-/Copy-Install-Portabilität herstellen und testen.
Tests	20 funktionale Fälle plus sechs Linter-Templates; erfolgreiche V3-Ausführung nur berichtet.	[C01] [C12]	Bestehende Suite erweitern, neue CI-Ausführung separat nachweisen.
Lint / CI / Coverage	Linter vorhanden, Copyright übersprungen, Legacy-Befunde; CI/Coverage fehlen.	[C10] [C12]	Sinnvolle Lintkorrekturen und reproduzierbare Automation aufbauen.
Dokumentation	README/INSTALL, Architektur, Konfiguration, bekannte Probleme, Implementierungs-/Referenzberichte.	[C04] [C10] [C12]	Aktualisieren und releasebezogene Lücken schließen; Bestand nicht als Neuschöpfung zählen.
Lizenzierung	Widersprüchliche Deklaration, Template-Notices, main-Copyrightheader; kein Projektlizenztext.	[C01] [C13]	Rechte/Grant und Kartendaten klären; keine vorhandene Gesamtfreigabe voraussetzen.
Distribution / Release	Humble dokumentiert, Basisinstallation vorhanden; keine Tags/Releases/Changelogs/Policies, keine Multi-Distro-Matrix.	[C04] [C12] [C13]	Index-/Binärrelease-Kriterien gesondert verifizieren; Kompatibilität und Pflegezusagen nachweisen.

Tabelle 10: Ausgangszustand und Bezug zur künftigen Forschung. Die Tabelle beschreibt keine abgeschlossenen Forschungsänderungen.

10.3 Zuordnungsregeln und offene Belege

Vorhandene Algorithmen, V2-Workflowbereinigung und sämtliche V3-Konfigurations-/Odometry-/Launch-/Testarbeiten sind vor der Baseline abgeschlossen. Ihre fachliche Herkunft vor dem Root-Import ist nicht bestimmbar. Spätere Änderungen dürften nur dann dem Forschungsvorhaben zugerechnet werden, wenn Zweck und Attribution belegt sind; ein neuer Zeitstempel oder ähnlicher Branchname genügt nicht.

Nicht bestimmbar bleiben Projekt-/Kartendatenrechte, nicht dokumentierte frühere Entwicklungsstände, entfernte/private Historien, unabhängig bestätigte V3-Hardwareleistung und weitergehende Distribution-/Plattformkompatibilität. Die vorhandenen Unterlagen beweisen weder die Unmöglichkeit eines funktionierenden Humble-Deployments noch vollständige Release-Eignung. Diese Grenzen müssen auch in einem späteren Vorher/Nachher-Vergleich bestehen bleiben.

11. Identifizierter Entwicklungsbedarf

11.1 Bestehender Stand, Defizit und geplante Änderung

Die folgenden Maßnahmen sind Empfehlungen für nachfolgende Arbeit. Sie sind weder durch dieses PDF umgesetzt noch vollständig normativ als ROS-Releasepflicht geprüft. Historisch belegte Defizite sind von Konventionen, Qualitätszielen und optionalen Verbesserungen zu trennen. Eine gesonderte aktuelle Kriterienprüfung ist erforderlich, bevor source indexing und Bloom/Buildfarm verbindlich bewertet werden können.

Arbeitsfeld	A: bestehende Implementierung	B: vorhandenes Defizit	C: künftige Modifikation / Nachweis
Paketkonformität / Metadaten	Python-Pakete, Namen, Manifeste, Installationshaken.	Versionswiderspruch, TODO-Beschreibung, unvollständige Metadaten.	Versionen/Metadaten konsistent festlegen; Paketprüfung am gewählten Zielstand.
Dependencies	Deklarationen und dokumentierte Systeminstallation.	Open3D/NetworkX und einzelne direkte ROS-Imports fehlen; Workarounds.	Direkte Dependency-Abbildung korrigieren, rosdep in sauberer Umgebung belegen.
Open-Source-Lizenz	Setupstring und Template-/Copyrightnotices.	Ungeklärter Projektgrant, Datenherkunft und Noticepflichten.	Rechtekette/Kartenrechte prüfen; autorisierten Lizenztext und konsistente Angaben schaffen.
Build / Installation	Scripts, Marker, Launch-/Config-Installation.	Karten nicht installiert; Source-/Symlink-only-Konfiguration.	Install-Prefix-unabhängige Ressourcen und Copy-Install-Verhalten entwickeln; Clean-Build/Install testen.
ROS-Interfaces / Launch	Standardtypen, Odometry/TF, vier Launches.	Absolute/relative Mischung, narrow-Orientierung, Action-/Snapping-Grenzen.	Stabile dokumentierte Verträge, Remapping/Namespaces und gezielte Grenzfalltests; Änderungen mit API-Auswirkung separat begründen.
Testing / Lint	20 Fälle, Linter-Templates, Validierungen.	Lücken bei Algorithmen, Cloudlayout, Controller, Installationen; Skip/Legacy-Lint.	Repräsentative Regressionen und sinnvolle Lintkorrekturen; vorhandene Tests beibehalten.
CI/CD	Dokumentierte manuelle Prüfungsschritte.	Kein automatisierter Clean-Build/Test/Testresult-Prozess.	Nach Dependency-/Installkorrekturen CI für rosdep, Build, Tests, Ergebnisse und ausgewählte Lints schaffen.
Dokumentation / Pflege	Umfangreiche Nutzungs-/Architektur-/Configdokumentation.	Fehlende Release-/Version-/API-/Supportzusagen, Changelog und Beiträgeprozess.	Bestand aktualisieren; proportionale Wartungs-, Beitrags- und Stabilitätsdokumente ergänzen.
Kompatibilität	Humble-Ziel und historische Berichte.	Keine belegte Multi-Distro-/Multi-Plattform-Ausführung.	Ziele explizit wählen, Matrix ausführen, nur beobachtete Inkompatibilitäten anpassen.
Releasevorbereitung	Paketgrundlage und nachvollziehbare Git-Stände.	Lizenz-/Metadaten-/Dependency-/Installationsfragen ungelöst.	Source-Indexing und binäres Release separat prüfen; Changelog/Version/Release-Rehearsal nach Voraussetzungen.

Tabelle 11: Bestehendes, Defizite und geplante Arbeit. Quellenbasis: [C01] [C05] [C10] [C12] [C13].

11.2 Verifikationsbedarf und Qualitätsspielraum

Lizenz-/Weiterverteilungsrechte, Paketmetadaten, Dependency-Auflösung und funktionsfähige installierte Ressourcen sind grundlegende Voraussetzungen für eine belastbare Distributionsvorbereitung. Welche einzelnen Metadaten-/Policydateien und Prüfungen formal für eine gewünschte ROS-Veröffentlichungsart gelten, wurde mit dieser reinen Historienkonsolidierung nicht neu recherchiert. Es wird daher kein Ready-Status und kein REP-2004-Quality-Level behauptet.

Namespaces, dokumentierte Schnittstellen und sinnvolle Launch-Hierarchien sind Integrationskonventionen beziehungsweise Nutzbarkeitsziele. Coverage, statische Analyse, Formatierung, Security-/Support-/EOL-Policies und Stabilitätszusagen sind nach angestrebter Qualität und Pflegekapazität zu bewerten. Nicht jedes fehlende Qualitätswerkzeug ist eine zwingende Releasebedingung. Umgekehrt ersetzt ein README keine geprüfte Installation oder Lizenzfreigabe.

Die historische Analyse liefert keine ausreichende Basis, Lizenzkompatibilität von Bibliotheken, garantierte ABI-Stabilität oder vollständige Jazzy-/Neuerversion-Unterstützung anzunehmen. Diese Punkte erfordern gezielte Folgeprüfung. Live-Mapping und neue Navigationsalgorithmen sind nicht Teil des erforderlichen Forschungsvorhabens; Algorithmusänderungen sind nur bei konkret begründeten Vertrags-/Fehlerbefunden einzuordnen.

11.3 Abhängigkeitsgerechte Reihenfolge

Zuerst sollten autorisierte Lizenz-/Datenprovenienz und korrekte Paketmetadaten/Dependencies geklärt werden. Darauf folgen reproduzierbarer Clean-Build und installiertes Ressourcen-/Konfigurationsverhalten. Erst auf dieser Grundlage lassen sich öffentliche Interface-/Konfigurationsverträge stabilisieren und mit gezielten Regressionen absichern. Vorhandene Tests werden erweitert, nicht als neu erfundene Grundlage ersetzt.

CI automatisiert anschließend die stabilisierten Dependency-, Build-, Install- und Testschritte. Danach werden die gewählten Distributions-/Plattformziele verifiziert. Releasebezogene Wartungsdokumentation, Source-Indexing- und Bloom-Vorbereitung folgen auf die technischen/rechtlichen Voraussetzungen. Dokumentation begleitet jeden Schritt; rein kosmetische Verbesserungen begründen keine Vorziehung gegenüber grundlegenden Blockern.

12. Schlussfolgerung

R3D-Planner erscheint im zugänglichen Git-Verlauf bereits als entwickeltes kartenbasiertes Verarbeitungssystem. Der Import enthält Traversierbarkeitsanalyse, beide Persistenzwege, Graphrekonstruktion, A*, ROS-Ausgaben und Navigationshelfer. Die kurze sichtbare Commit-Chronologie beschreibt deshalb vor allem Übernahme, Umorganisation, Workflowbereinigung und Integrationsfortentwicklung, nicht die gesamte ursprüngliche Entstehung der Algorithmen.

V1 bewahrt duale Datenwege, V2 reduziert auf die vorhandene PCD-Pipeline. V3 schafft eine zentrale validierte Konfiguration, wählbare Roboterverträge, installierte Subsystemlaunches, standardisierte Odometry-/TF-Pose und fokussierte Funktionstests. main ergänzt ausschließlich Kommentarheader. Diese technische Reife ist real vorhandener Sourceumfang; umfassende Algorithmusregression, generische Installation und hardwarebezogene Systemzuverlässigkeit sind dadurch nicht vollständig nachgewiesen.

Die eindeutige Referenz des Forschungsvorhabens bleibt c65642fe56015556f696364a724cb5b976a27888 mit Bewertungsstichtag 12. Oktober 2026. Alle darin enthaltenen Funktionen und V3-Integrationsleistungen sind Vorbestand. Der spätere Forschungsbeitrag muss an nachvollziehbaren Verbesserungen von Lizenzierung, Metadaten, Dependencies, Installation, Schnittstellen, Tests, CI und Releasevorbereitung gemessen werden.

Der Übergang zur künftigen Arbeit ist damit klar: vorhandene Funktionalität erhalten und belegbar distributierbar machen, offene Rechts-/Portabilitätsfragen klären und Qualitätsnachweise erweitern. Dieser Bericht dokumentiert den Ausgangspunkt. Er setzt keine ROS-Änderung um und erklärt weder ein Release noch eine Lizenzentscheidung für abgeschlossen.

Anhang A. Commit-Übersicht und Chronologie

Alle 13 eindeutigen Commits sind in tatsächlicher Zeitreihenfolge aufgeführt. In der Voranalyse stimmen Author- und Committer-Zeitstempel überein; beide Identitäten lauten durchgehend Basti <baau1001@stud.hs-kl.de>. Das ist Git-Metadatum, kein Beweis umfassender Rechteinhaberschaft. Zeiten sind +02:00. Originalsubjects sind zur Reproduzierbarkeit unverändert wiedergegeben.

C01 | 2026-09-14T16:44:14+02:00

`ca9f44930a44391577c8fd822c0d7f557fcd9da4`

Originalsubject: Added R3D-Planner source code and test maps
Einordnung: Root-Import: zwei Pakete, beide Pipelines, Navigation, Tests, Karten und Workspace-Artefakte.

Parent: ROOT (kein Parent).

C02 | 2026-09-14T16:55:12+02:00

`ce4af0b9ac0dcf751b0b858e019ffc3ea042e3e0`

Originalsubject: Added newest Version of R3D-Planner
Einordnung: Flaches Layout, Workspace-/Shell-Entfernung; vorübergehende PCD-Auslassung.

Parent: [C01] ca9f44930a44391577c8fd822c0d7f557fcd9da4.

C03 | 2026-09-14T17:00:00+02:00

`a24a3d3a18930b678359056ee69f18e9940bb0f5`

Originalsubject: Added PCD support
Einordnung: Wiederherstellung der Root-PCD-Quelle, RGB-/Alpha-/Shutdown-Verhalten.

Parent: [C02] ce4af0b9ac0dcf751b0b858e019ffc3ea042e3e0.

C04 | 2026-09-14T23:33:20+02:00

`253ff81110561bb20c83cb66569b71eca8f665b1`

Originalsubject: docs: document repository architecture and setup
Einordnung: README/INSTALL/Architektur/Probleme; gemeinsamer Verzweigungspunkt.

Parent: [C03] a24a3d3a18930b678359056ee69f18e9940bb0f5.

C05 | 2026-09-14T23:41:37+02:00

`337bbd3d2b956f58a87d17e3c1c77e812cd333ac`

Originalsubject: refactor: create PCD-only V2 workflow
Einordnung: V2: Pickle-Module/Scripts/Bytecode und vier Dependency-Einträge entfernt.

Parent: [C04] 253ff81110561bb20c83cb66569b71eca8f665b1.

C06 | 2026-09-15T00:22:43+02:00

`3e5360a94e5783ed6ef09426ed5d66175f3097c3`

Originalsubject: docs: rewrite documentation in maintainer voice
Einordnung: V2: neun Dokumente in Maintainer-Stil umgeschrieben.

Parent: [C05] 337bbd3d2b956f58a87d17e3c1c77e812cd333ac.

C07 | 2026-09-15T00:22:47+02:00

`7a4c0cfda336763e6b32d69718e43d38eef5fb7c`

Originalsubject: docs: rewrite documentation in maintainer voice
Einordnung: V1: parallele, patchverschiedene Maintainer-Umschreibung.

Parent: [C04] 253ff81110561bb20c83cb66569b71eca8f665b1.

C08 | 2026-09-15T00:42:36+02:00

`0306f8f4f3bb88e0b3fa76ca5bf5093d6aae7b9a`

Originalsubject: docs: turn README into usage guide
Einordnung: V1: README als Nutzungsanleitung.

Parent: [C07] 7a4c0cfda336763e6b32d69718e43d38eef5fb7c.

C09 | 2026-09-15T00:42:37+02:00

`cf0c527b587b387c56c69ae4c90a98d180f2d22d`

Originalsubject: docs: turn README into usage guide
Einordnung: V2: README als Nutzungsanleitung; anderer Patch.

Parent: [C06] 3e5360a94e5783ed6ef09426ed5d66175f3097c3.

C10 | 2026-09-15T16:01:50+02:00

`3abd002e8cc9e7a26cacaf43a5c20b1822664f14`

Originalsubject: docs: validate reference and anonymize paths
Einordnung: V2-Tip: Referenzbericht, Pfade und dokumentierte Integrationskorrekturen.

Parent: [C09] cf0c527b587b387c56c69ae4c90a98d180f2d22d.

C11 | 2026-09-15T16:02:04+02:00

`01348f3285a0e7582f70cc5f2ed3d1bf4b7f904b`

Originalsubject: docs: anonymize workspace paths
Einordnung: V1-Tip: Beispielpfade in drei Dokumenten anonymisiert.

Parent: [C08] 0306f8f4f3bb88e0b3fa76ca5bf5093d6aae7b9a.

C12 | 2026-09-16T15:08:56+02:00

`3752404a08867e9e6f3725300b88e148d4d5be37`

Originalsubject: Introduce V3 robot configuration and standard odometry interface
Einordnung: V3: YAML/Launch/Odometry, Parameter, Validierung, Tests und Dependencies.

Parent: [C10] 3abd002e8cc9e7a26cacaf43a5c20b1822664f14.

C13 | 2026-09-17T00:56:32+02:00

`c65642fe56015556f696364a724cb5b976a27888`

Originalsubject: docs: add standardized author and repository source headers
Einordnung: main: 18 Kommentarheader, kein AST-/Laufzeitunterschied.

Parent: [C12] 3752404a08867e9e6f3725300b88e148d4d5be37.

Anhang B. Reproduzierbarkeit und Branch-Nachweise

B.1 Vergleichsregeln

Die Kommandos verwenden vorhandene Objekte/Refs und benötigen keinen Checkout, Fetch, Merge oder Schreibzugriff auf die Projektquelle. Pfade sind im Folgenden relativ zum R3D-Planner-Repository. Historische Root-Quellen tragen zusätzlich das Präfix ros2_r3d_planner_ws/src/. Generierte build/lib-/install-Kopien sind keine bevorzugte Implementierungsevidenz.

```
git --no-optional-locks log --branches --remotes --graph --decorate --oneline
git --no-optional-locks rev-list --branches --remotes --count
git --no-optional-locks merge-base origin/V1 origin/V2
git --no-optional-locks rev-list --left-right --count origin/V1...origin/V2
git --no-optional-locks diff origin/V1 origin/V2 -- r3d_planner r3d_preprocessor
git --no-optional-locks diff origin/V2 origin/V3
git --no-optional-locks diff origin/V3 main
git --no-optional-locks show <vollstaendige-SHA>:<historischer-Pfad>
git --no-optional-locks diff <Parent-SHA> <Commit-SHA> -- <Pfad>
```

Ein git rev-list --all ^<Baseline> liefert auch ältere V1-Sidebranch-Commits und ist kein Test auf spätere Entwicklung. Für jeden Kandidaten müssen Abstammung und Datum getrennt betrachtet werden: git merge-base --is-ancestor <Baseline> <Kandidat>. Die zugrunde liegende Analyse fand keinen zugänglichen Nachfolger der Baseline.

B.2 Stabile Patch-IDs und Äquivalenzen

[C06] Maintainer voice, V2:

`4f82a74e3f214b3e66593c75ac45e6d48c638dbe`

[C07] Maintainer voice, V1:

`86dce27432b9bac74fb71b73cf5f34759caee9c4`

[C08] Usage guide, V1:

`b1f25e9da2b039c0245e8d8b7307758e63e3e669`

[C09] Usage guide, V2:

`fbd1a584d7e441d8dca75c6e4ad916c0bee4d30c`

[C10] Reference/path docs, V2:

`6ef57a7e826519bd5293f01550e50ea9267795f9`

[C11] Path docs, V1:

`d85682161f82c4617c2adb99f4ec805e23dc84cf`

```
git --no-optional-locks show <SHA> --format= | git patch-id --stable
```

Die ungleichen IDs belegen keine patchidentischen Cherry-Picks zwischen gleichnamigen Dokumentationsänderungen. Davon getrennt sind die Source-Äquivalenzen: 31 kanonische Root-/Restoration-Textdateien byteidentisch; erhaltene V1-/V2-Funktionsquellen gleich; alle 18 V3-/main-ASTs gleich. Dadurch werden Wiederherstellungen und Headeränderungen nicht als neue Algorithmusentwicklung gezählt.

Anhang C. Quellpfade und Funktionsnachweise

Die Pfade beziehen sich auf die flache Struktur ab [C02]/[C03]. Für [C01] ist ros2_r3d_planner_ws/src/ voranzustellen. Nicht mehr vorhandene Dateien sind über ihren historischen Commit abrufbar. Kurze Modulnamen in den Fachkapiteln werden hier vollständig zugeordnet.

r3d_preprocessor/r3d_preprocessor/r3d_pcd_analyser.py

[C01] [C03] [C12] VoxelTraversabilityAnalyzer: load_and_voxelize, augment_floor_from_density, check_collision_cylinder, identify_walkable_nodes, _fill_floor_planes, identify_stairs, save_colored_pcd. PcdToGraphNode: Parameter und run_processing.

r3d_preprocessor/r3d_preprocessor/r3d_pcd_to_graph.py

[C01] [C05] Historisch VoxelNavigationGraph: build_graph/save_graph, ähnliche Analyse-/Clearance-Funktionen. In V2 entfernt.

r3d_planner/r3d_planner/r3d_pcd_path.py

[C01] [C03] [C12] GlobalGraphPlanner: load_and_rebuild_graph_from_pcd, get_nearest_node, heuristic, a_star, reconstruct_path, execute_callback, publish_visual_path.

r3d_planner/r3d_planner/r3d_global_planner.py

[C01] [C05] Historischer Pickle-Planner: load_graph, A*, Action und CUBE_LIST-Darstellung; in V2 entfernt.

r3d_planner/r3d_planner/r3d_local_filter.py

[C01] [C12] ObstacleCliffFilter: Konstruktor/pc_callback, packed-XYZ, Hindernis-/Stufen-/Klippenoutputs.

r3d_planner/r3d_planner/r3d_path_follower.py

[C01] [C12] PathFollower: path_callback, obstacle_callback, get_robot_pose, control_loop; Nullbefehl bei Poseverlust seit V3.

r3d_planner/r3d_planner/r3d_rviz_interface.py

[C01] [C12] RVizInterfaceNode: Point-/Initial-/Goalcallbacks, recalibrate_callback, publish_map_odom_tf, send_path_request.

r3d_planner/r3d_planner/odometry.py

[C12] OdometryInput.callback/map_pose, quaternion_product/components; keine TF-Publikation.

r3d_planner/r3d_planner/configuration.py

[C12] UniqueLoader, default_config_path, read_yaml, keys/number/text, ros_parameters, load_configuration.

r3d_planner/r3d_planner/launching.py

[C12] configured_nodes/description; vier Modi, OpaqueFunction, IfCondition und Parameterweitergabe.

r3d_planner/config/planner_config.yaml; r3d_planner/config/robots/Go2W.yaml

[C12] Zentraler Selector/shared-/Komponentenvertrag und Robotgeometrie/Odometry/Topics; Legacy r3d_planner_params.yaml schon [C01].

r3d_planner/launch/*.launch.py

[C12] planner.launch.py, preprocessor.launch.py, map.launch.py, follower.launch.py.

r3d_preprocessor/r3d_preprocessor/r3d_pcd_publisher.py

[C01] [C03] [C12] PCDPublisher.create_pc2_msg: aktive RGB-/QoS-/Frameausgabe.

r3d_preprocessor/r3d_preprocessor/r3d_pcd_voxel_publisher.py

[C01] [C03] Alternativer XYZ-Publisher ohne Console-Entry; fester Frame/gleicher Node und Topic.

r3d_preprocessor/r3d_preprocessor/r3d_voxel_map_publisher.py

[C01] [C05] Historischer Pickle-Marker-Publisher, in V2 entfernt.

r3d_planner/r3d_planner/r3d_path_test.py

[C01] [C03] PathTestTFNode.publish_fake_robot_tf; statischer manueller Helfer, kein Odometry-Publisher.

r3d_planner/test/test_configuration.py

[C12] [C13] Neun Funktionen, 20 Fälle; Headeränderung bei main. Weitere Linter-Templates in beiden test-Verzeichnissen [C01].

r3d_planner/{package.xml,setup.py,setup.cfg}; r3d_preprocessor entsprechend

[C01] [C05] [C12] Manifest-/Setup-Metadaten, Script- und Ressourceninstallation, Dependency-Diffs.

docs/ARCHITECTURE.md; docs/CONFIGURATION.md; docs/KNOWN_ISSUES.md

[C04] [C12] Architektur, Konfiguration, Grenzen; README_AUDIT.md dokumentiert Textpflege.

docs/V3_IMPLEMENTATION.md; docs/REFERENCE_COMPARISON.md

[C10] [C12] Berichtete Testergebnisse und historischer V2-Kontext. Externe Aussagen nicht erneut verifiziert.

r3d_planner_launch.sh im Root-Snapshot

[C01] [C02] Hardcodierter Workspace, sechs ros2-run-Prozesse, Prozess-Trap; bei Flattening entfernt.

Anhang D. Quellenkonsolidierung und Grenzen

D.1 Primärdokumente und inhaltliche Zuordnung

Alle elf Markdown-Primärdokumente wurden vollständig eingelesen. Der Bericht übernimmt ihre relevanten technischen/historischen Befunde, zählt wiederholte Erklärungen einmal und bewahrt vollständige Commitnachweise. Die folgende Zuordnung dient als Inhaltskontrolle vor der ausdrücklich angeforderten Entfernung der ursprünglichen Dateien. SHA-256-Präfixe bezeichnen die bei der Konsolidierung gelesenen Fassungen.

01_REPOSITORY_HISTORY.md

Repository, Topologie, Root-/Baseline-Bestand und Reproduktion: Kapitel 2/3, Anhang B/D. Prüfsummenpräfix: 55568545322947d0.

02_VERSION_TIMELINE.md

Alle Commitdaten/Parents/Subjects: Kapitel 3 und Anhang A. Prüfsummenpräfix: 88ceef990af91a82.

03_VERSION_COMPARISON.md

V1/V2/V3/main-Fähigkeiten, Änderungen, Tests/Grenzen: Kapitel 3/9. Prüfsummenpräfix: efddbf07e040e401.

04_FUNCTIONAL_EVOLUTION.md

Algorithmen und Navigation: Kapitel 4/5, Nachweise Anhang C. Prüfsummenpräfix: e299e86e0a82ab73.

05_ROS_INTEGRATION_HISTORY.md

Interfaces, TF/Odometry, Launch/YAML/Installation: Kapitel 6. Prüfsummenpräfix: 0efb500c8e109838.

06_TESTING_AND_QUALITY_HISTORY.md

Tests, Ergebnisse, Umgebung und QA-Lücken: Kapitel 7. Prüfsummenpräfix: f780c24db0a24546.

07_DEPENDENCY_AND_LICENSE_HISTORY.md

Metadaten/Dependencies, Notices, Provenienz und offene Rechte: Kapitel 8. Prüfsummenpräfix: 8dfe9166ae3c57a7.

08_FORK_AND_BRANCH_COMPARISON.md

Branch-/Featurematrix, Patch-Identität und Scope: Kapitel 3/9, Anhang B/D. Prüfsummenpräfix: 7f14de2c7c7d926d.

09_RESEARCH_BASELINE_COMPARISON.md

Vorbestand/Defizite/Forschungszuordnung und Reihenfolge: Kapitel 10/11. Prüfsummenpräfix: b05d7b2fe73ce931.

10_COMPLETE_CHANGELOG.md

Alle substantiellen Änderungen inklusive Dokumentationscommits: Kapitel 3, Anhang A. Prüfsummenpräfix: 6a2961feae172a81.

README.md

Zweck, Evidenzregeln, kritische Zuordnung und Grenzen: Kapitel 1/10, Anhang D. Prüfsummenpräfix: 2b48ab7e578fa922.

D.2 Administrative Bestandsinformationen

Die Voranalyse dokumentiert origin als git@github.com:RoboProjekt/R3D-Planner.git für Fetch/Push, main als lokalen Branch und vier origin-Trackingbranches. Das Repository war nicht shallow; ein Root und keine Merges waren vorhanden. fsck meldete keine unerreichbaren Objekte. refs/codex/turn-diffs verweist auf den Baseline-Tree 2428d98a2b1a018275cb1682ca5193eadbe86c48 und stellt keine zusätzliche Commitlinie dar.

Historisch erfasste Remote-Metadaten: öffentliches Repository, default_branch main, fork=false, kein Parent/Source, forks_count/network_count jeweils 0, leere Fork-/Tag-/Releaselisten. created_at war 2026-09-14T14:40:37Z, pushed_at 2026-09-16T22:56:40Z, license null. Diese Angaben sind eine übernommene zeitgebundene Bestandsaufnahme der Voranalyse; für die Konsolidierung wurde keine neue Remote-/Fork-Untersuchung durchgeführt.

Private, gelöschte oder nicht bekanntgegebene Historien sind nicht bestimmbar und wurden nicht als analysiert ausgegeben. Andere Repositories werden weder verglichen noch als Nachfolgerbeitrag zugerechnet. Original-Algorithmusdaten vor Root, Rechtsherkunft, generische Hardwareleistung und Multi-Distribution-Unterstützung bleiben in den Fachkapiteln entsprechend eingeschränkt.

D.3 Umfang der Dokumentoperation

Die PDF-Erstellung ändert keinen Quellcode, keine Dependencies, Branches, Remotes oder Git-Historie. Es werden keine Commits oder Pushes erzeugt. Die elf generierten Primär-Markdown-Dateien dürfen erst nach erfolgreicher Inhalts-, Referenz- und PDF-Layoutprüfung entfernt werden; andere Dokumentation bleibt erhalten. Die Verifikation prüft dies gegen die zu Beginn erfassten Dateiprüfsummen. Das PDF bewahrt die historische Information als konsolidiertes Enddokument.

Formale PDF-Prüfung und Sichtkontrolle sind keine zusätzlichen ROS-Funktionstests. Aus dem Vorliegen dieses Berichts dürfen weder abgeschlossene Implementierungsarbeit noch Releasefreigabe oder eine bestimmte Paketqualität abgeleitet werden.